

P2P Decentralised Lending Service

Project Report (Spec §2)

Federico Lombardi and Francesco Polperio

May 21, 2026

1 Implementation Choices

Two Contracts Architecture (`contracts/LendingPool.sol`, `contracts/LoanContract.sol`): We separated the logic into `LendingPool` and `LoanContract` to fulfill requirement [R2] ("new active loan must be managed by a newly deployed dedicated smart contract"). This isolation ensures that each loan is an independent state machine, preventing complex cross-contamination of locked funds and repayment calculations, albeit at the cost of higher gas usage during loan deployment.

UUPS Upgradability: We chose UUPS (EIP-1822) over Transparent Proxy. UUPS is more gas-efficient because the upgrade logic resides in the implementation contract rather than the proxy. Access control is strictly enforced by overriding `_authorizeUpgrade` with the `onlyOwner` modifier.

Termination Pattern: EIP-6049 marks `selfdestruct` as deprecated, and post-Cancun it no longer clears storage. Instead, we use a `terminated` boolean flag in `LoanContract`. This completely disables state changes safely and semantically identically, without relying on deprecated EVM features.

Capital + Interest Settlement (`LoanContract.sol::partialRepay`): A loan becomes `Successful` only when both `remainingLoanAmount` and `remainingInterest` reach zero before `expiryBlock`. Each `partialRepay` processes the incoming `msg.value` in three ordered stages: (1) capital is distributed via waterfall in descending `initialLocked` order; (2) any `afterBase = msg.value - baseAmount` feeds the interest loop, capped at the residual `expectedInterestOf[c]` for each contributor and split internally into a per-contributor `collateral share` (sent to the comp pool) and `gain share` (credited directly to the contributor's wallet via `lendingPool.creditInterest`); (3) any leftover beyond capital and expected interest is `excess` and goes *entirely* to the compensation pool. The applicant's promised `interestRate` is therefore enforced on-chain rather than being a non-binding declaration.

Per-Contributor Interest Tracking (`LoanContract.sol` storage): `expectedInterest = loanedAmount * interestRate / 100` (immutable). Per-contributor quotas `expectedInterestOf[c] = expectedInterest * initialLocked[c] / loanedAmount` (proportional). `interestPaidGrossOf[c]` accumulates the gross interest already paid to each contributor across multiple installments. `remainingInterest = sum(expectedInterestOf - interestPaidGrossOf)` provides the global cap, ensuring that overpaying contributors is impossible and the residue routes deterministically to the comp pool.

Failed-Loan Interest Forfeit (`markFailed`, `requestCompensation`): On the `Active` → `Failed` transition, `remainingInterest` is set to 0. From that point on, the interest loop in `partialRepay` is a no-op: any further `afterBase` becomes `excess` and is forwarded entirely to the comp pool. Contributors that defaulted on a loan therefore never receive interest after the loan is failed, regardless of how much the applicant repays later.

Proportional Base Forfeit (`LoanContract.sol::splitBaseForfeit`): When a contributor has already drawn compensation from the comp pool (`alreadyCompensated[c] >`

0), each subsequent capital chunk `take` destined to `c` is split: a portion `toComp = take * unrecoveredAdvance / remainingShare` goes back to the comp pool (to recover the advance), and the remainder `toC` is refunded to the contributor’s deposit position. Under waterfall saturation the math is exact (`toComp == unrecoveredAdvance`), so the comp pool is fully repaid the advance with no dust.

Weighted Vote on Disposable: Voting weight is dynamically evaluated as `disposableValue(c)` at the moment the proposal is resolved, not when submitted. A contributor withdrawing funds during the voting period automatically reduces their voting power, ensuring they always have “skin in the game”.

Proportional Lock: When a loan is approved, locked shares are calculated as `floor(amount * disposable_c / totalDisposable)`. Any precision leftover from the floor division is explicitly deducted from the loaned amount and not credited to the applicant, preserving perfect accounting invariants.

Repayment Distribution Order: Base capital is repaid sequentially in descending order of `initialLocked`. A deterministic tie-breaker (ascending by contributor address) guarantees an unambiguous waterfall order.

Reentrancy Guards & CEI: The CEI (Checks-Effects-Interactions) pattern is strictly applied. All state updates precede external calls. As a defense-in-depth measure, a manual mutex guard (`_reentrancyStatus`, since OpenZeppelin v5 dropped `ReentrancyGuardUpgradeable`) is placed on `deposit`, `withdraw`, and `resolveProposal`. The new `compensateFromPool` (see below) no longer performs an external transfer, so the mutex on it has been removed accordingly.

Compensation Accounting — Interpretation B (`LendingPool.sol::compensateFromPool`): The spec sentence “*the contributor receives from the compensation pool enough value as to recoup the value locked by them*” was confirmed by the professor to be implemented as *credit-to-deposit* rather than transfer to the contributor’s EOA. The compensation amount stays inside the `LendingPool`; only `compensationPool`, `lockedValue[c]` and `totalLocked` are decremented. The contributor’s `deposits` entry is unchanged, so their `disposable = deposits - lockedValue` grows by the compensated amount: they can then `withdraw` or reuse the funds for voting on future proposals. The per-loan ledger is unaffected: `alreadyCompensated` still tracks payouts monotonically, and `compRecovered` still tracks how much the applicant has repaid back to the pool, so the proportional base-forfeit in `_splitBaseForfeit` continues to work unchanged. The ETH invariant `poolBalance = sum(deposits) - totalLocked + compensationPool` is preserved by construction.

Oracle Fee Constant (`BitcoinOracle.sol::MIN_ORACLE_FEE`): Per spec, the minimum oracle fee equals *gas cost of `update()` × 0.1 gwei*. Measured gas $\approx 45,667$, yielding a constant of $4,566.7 \text{ gwei} = 4.5667 \times 10^{12} \text{ wei}$ hardcoded into the oracle contract.

2 Gas Evaluation Results

The following table summarizes the on-chain gas measurements for 20 critical operations, measured at a 1 gwei gas price during benchmarking (distinct from the 0.1 gwei multiplier used in the `MIN_ORACLE_FEE` constant). Raw data: `data/gas_report.csv`.

Costliest Operation: `resolveProposal` (Approved) is the most expensive operation ($\sim 1.8\text{M}$ to 2.6M gas) as it involves deploying the `LoanContract` and looping through contributors. The gas cost scales linearly with N contributors.

Cheapest Operations: `withdraw` ($\sim 50\text{k}$ gas) and `vote` ($\sim 60\text{--}120\text{k}$ gas).

First-call overhead: `requestCompensation` drops from 115k to 67k gas on subsequent calls because the first call performs additional state transitions absent later: it flips `status` to `Failed` (cold SSTORE), invokes `increaseCollateral` on the pool (extra SSTORE + event), and emits `MarkedFailed`. Subsequent claims hit only the comp-pool payout path with warm slots.

Operation	Scenario	Gas Used	Cost (ETH at 1 gwei)
deposit	new contributor	141,091	0.000141
deposit	existing contributor	42,421	0.000042
withdraw	partial	49,984	0.000049
requestOracleUpdate	via pool forward	40,973	0.000040
submitProposal	valid (amount ≤ disp, btc ok)	184,398	0.000184
vote	approve	123,903	0.000123
vote	reject	59,675	0.000059
resolveProposal	Approved (N=2)	1,857,149	0.001857
resolveProposal	Approved (N=5)	2,162,796	0.002162
resolveProposal	Approved (N=10)	2,678,306	0.002678
resolveProposal	Rejected (pool low)	63,139	0.000063
resolveProposal	Rejected (btc liquidity)	73,084	0.000073
resolveProposal	Rejected (weighted vote)	82,345	0.000082
partialRepay	mid (no overpay)	78,805	0.000078
partialRepay	close Successful	152,272	0.000152
partialRepay	overpay (extra interest)	155,052	0.000155
requestCompensation	first call (marks Failed)	114,994	0.000114
requestCompensation	subsequent (refill claim)	67,444	0.000067
partialRepay	on Failed loan (proportional split)	147,364	0.000147
upgradeToAndCall	UUPS v2 swap	37,736	0.000037

3 Reentrancy Modification

To demonstrate the vulnerability of improperly ordered state updates, we introduced `LendingPoolVulnerable`.

Vulnerable Code Snippet (withdraw):

```
function withdraw(uint256 amount) external {
    require(disposableValue(msg.sender) >= amount, "Insufficient");
    (bool ok, ) = msg.sender.call{value: amount}(""); // CEI Violation
    require(ok, "Transfer failed");
    unchecked {
        deposits[msg.sender] -= amount; // State updated AFTER call
        totalFundingPool -= amount;
    }
}
```

Attacker Contract Snippet (ReentrancyAttacker.sol):

```
receive() external payable {
    if (attackCount < MAX_REENTRIES && address(pool).balance >= attackAmount) {
        attackCount++;
        pool.withdraw(attackAmount); // Recursive re-entry
    }
}
```

Attack Result (from `test/Reentrancy.test.js`): Two honest contributors deposit 5 ETH each; the attacker deposits 1 ETH (pool holds 11 ETH). On `attack()`, `receive()` re-enters `withdraw(1 ETH)` five additional times. The attacker walks away with 6 ETH (initial 1 + 5 stolen); the pool drops from 11 ETH to 5 ETH. Alice can still withdraw her 5 ETH, but Bob’s subsequent `withdraw(5 ETH)` reverts with "Transfer failed" — concrete proof that 5 ETH of honest deposits were stolen.

Defenses Applied: In the production `LendingPool.sol`, we strictly follow the CEI pattern: `deposits` and `totalFundingPool` are decremented *before* the external ETH transfer. As defense-in-depth, a manual `nonReentrant` mutex (OpenZeppelin v5 removed `ReentrancyGuardUpgradeable`) guards critical functions. The same attack on the secure contract reverts with "Reentrant call" / "Transfer failed".

4 Malicious Strategies Discussion

The protocol design mitigates several attack vectors. Below is an analysis of potential malicious strategies.

Collusion (Applicant + Contributor): A contributor colludes with an applicant by approving a high-risk, high-interest loan. While possible, the voting weight equals the contributor's *disposable* ETH ("skin in the game"), and approval immediately locks a proportional share of that disposable into the loan. A colluding contributor therefore must commit their own real value to pass the proposal. If the applicant defaults, the contributor loses their locked share (only partially recoverable via the comp pool). The profit from the applicant's default is offset by the contributor's loss.

Withdraw Timing: Can a user withdraw before resolving a proposal to inflate their relative voting weight? No. Voting weight evaluates current `disposableValue(c)`. Withdrawing decreases your own disposable value and thus your own weight.

Front-run partialRepay with withdraw: An attacker tries to withdraw funds just before a large repayment arrives. This is ineffective because a contributor's share of repayment is fixed at loan creation (`initialLocked`). Withdrawing disposable funds does not alter the repayment fractions.

Sybil Contributors: Splitting deposits across multiple addresses to manipulate votes is futile because voting weight is proportional to ETH value, not address count.

Strategic Non-Voting: Abstaining from risky loans avoids locking funds, but since abstention counts as a "reject" (it does not contribute to the `weightedYes`), it is a rational, safe behavior rather than a malicious exploit.

Repayment Majority Rule Exploit: The waterfall repayment mechanism pays the largest contributor first. A strategic user could deposit a massive amount right before a promising loan to secure the top spot in the waterfall. If the applicant partially repays and then defaults, the large contributor is made whole while smaller ones lose out. This is a deliberate design trade-off, not a bug, but it represents the most profitable "selfish" strategy.

Loan Fail Intentional (Applicant = Contributor): An attacker acts as both applicant and contributor, defaults on the loan, and then requests compensation from the pool. While they recover some value from the compensation pool, doing so increases the global collateral percentage, penalizing the whole system. The profit is marginal and bound by the compensation pool limit, making it largely unprofitable against the locked ETH they forfeited.

Conclusion: The most realistic "selfish" strategy is securing the top waterfall spot via a massive deposit. Other vectors are either mathematically unprofitable or highly self-punitive.

5 User Manual

Prerequisites: Node.js ≥ 18 , Python ≥ 3.10 , geth (for the private chain). Install dependencies: `npm install`, then `python -m venv venv && source venv/bin/activate && pip install -r requirements.txt`.

1. Compile Contracts

`npx hardhat compile`

2. Start Local Chain

Start a geth node initialised from the provided genesis file:

```
geth --datadir chaindata --networkid <id> --http --http.api eth,net,web3,personal  
--allow-insecure-unlock
```

(Keep this terminal running. The prefunded genesis account may only transfer value to new accounts, never deploy.)

3. Initial Setup

`python scripts/InitialSetup.py` (Creates accounts, deploys proxy/implementation, funds wallets, saves data/*.json).

4. Start Oracle Service

Export the operator key, then run the off-chain oracle (keep running):

```
export OPERATOR_PRIVATE_KEY=0x... && python oracle/oracle_service.py
```

5. Run Auto-Voter (Optional)

`python scripts/AutoVoter.py` (Listens for `ProposalSubmitted` events and votes *approve* on every new proposal, per spec §1.5).

6. Run Demo

`python scripts/DemoOperations.py` (Executes the end-to-end lifecycle of loans).

7. Gas Measurement

`python scripts/GasMeasurement.py` (Generates `data/gas_report.csv`).

8. Run Test Suite

`npx hardhat test` (Runs all automated unit and integration tests).

9. Reproduce Reentrancy Demo

`npx hardhat test test/Reentrancy.test.js`